2017 Pattern Recognition Association of South Africa and Robotics and Mechatronics International Conference (PRASA-RobMech)
Bloemfontein, South Africa, November 29 - December 1, 2017

K-means-sharp: modified centroid update for outlier-robust k-means clustering

Peter O. Olukanmi
Institute for Intelligent Systems
Electrical & Electronics Engineering
University of Johannesburg.
peteroolukanmi@gmail.com

Bhekisipho Twala
Institute for Intelligent Systems
Electrical & Electronics Engineering
University of Johannesburg.
btwala@uj.ac.za

***Abstract*—The classical k-means clustering algorithm is easily misled by outliers. To address this problem, we modify its centroid update step so that outliers are avoided when new centroids are computed. Our modified algorithm, named k-means-sharp (k-means#), detects outliers automatically by means of a global threshold derived from the distribution of point-to-centroid distances. The approach requires neither user intervention nor prior knowledge of the number of outliers. Since it preserves k-means' structure, k-means# inherits the former's ease of implementation and if desired, it can benefit from other existing k-means improvements. On 10 datasets - 2 standard datasets and 4 outlier-contaminated versions of each - k-means# yields lower within-cluster mean squared error than k-means. It detects outliers with high accuracy (90.4% average and 99.2% maximum) and precision (86.8% average and 100% maximum), and moderate recall (52.4% average and 90.0% maximum) and F1-score (61.1% average and 92.9% maximum).**

Keywords—k-means; clustering; outlier; anomaly; detection; cost function; threshold

I. INTRODUCTION

Real-world datasets commonly contain noise and outliers. Due to the volume, velocity and variety of today's data universe, learning algorithms need to be inherently robust to such conditions, without the need for manual preprocessing. Some traditional machine learning algorithms do not satisfy this requirement: they assume the absence of outliers. An example is the k-means clustering algorithm. It is one of the most widely used clustering techniques [1], and has also been listed among the top 10 data mining algorithms [2].

Clustering is one of the most fundamental modes of learning [3][4]. Thus, it is widely studied in pattern recognition, machine learning and statistics. It is an unsupervised learning problem, in contrast to classification in which correct labels are available. The goal of clustering is to partition data into natural groups, such that similar instances belong in the same group while dissimilar ones belong in different groups [5]. Jain [4] notes that the vast literature related to clustering speaks to its importance, and it is difficult to either exhaustively list its numerous applications or to exhaustively review the thousands of related publications. Nevertheless, his work [4] is a widely cited attempt at a comprehensive review of clustering methods and issues. Among partitional clustering techniques, k-means is widely favoured due to its practicality, simplicity and efficiency [6]. It has been successfully applied in many fields. A few of these applications are document clustering [7], market segmentation [8], image segmentation [9], social media analysis for banking applications [10], feature learning [11], and wireless sensor network management [12].

However, some drawbacks are associated with the classical k-means algorithm, many of which are still being widely researched to date [13][14][5]. It requires the number of clusters to be supplied by the user; it is sensitive to the choice of initial centroids; and it only yields a local minimum rather than a global one. As earlier mentioned in this section, the k-means algorithm is extremely sensitive to outliers, being a least-squares method. Outliers are isolated or anomalous data that do not fit into the overall clustering pattern. They tend to skew k-means' cluster regionalization by contributing false large values to the objective function. Also, using them as initial centroids results in poor clustering. Unfortunately, k-means can neither detect these outliers nor differentiate them from the rest of the data. This calls for modifications to the classical algorithm, such as introduction of seeding techniques to avoid outliers being selected for initialization. A more comprehensive solution to the problems posed by outliers is to employ mechanisms for detecting them, so they can be excluded from the data supplied to the clustering algorithm. Some outlier detection methods are discussed in Section II of this paper.

Besides its role in making k-means robust, outlier detection has its own applications. As pointed out by Jin et.al. [15], from a knowledge discovery point of view, outliers are sometimes more interesting that the rest of the data. They contain information underlying anomalies in contrast to the common cases represented by the rest of the data. Fraud and intrusion detection are common examples of such applications in which outliers are more interesting than common patterns.

In this paper, we aim to preserve the structure, simplicity and efficiency of the classical k-means, while addressing its non-robustness to outliers. We propose an approach that only modifies the centroids update step in the classical algorithm. Instead of computing new centroids by finding the mean of all data points within each cluster, our modified algorithm detects and exempts putative outliers from this computation. A point is classified as an outlier if it is at a distance further than T from its current cluster centroid. The threshold T is computed by

978-1-5386-2313-8/17/$31.00 ©2017 IEEE

Authorized licensed use limited to: INDIAN INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on October 29,2026 at 14:37:17 UTC from IEEE Xplore. Restrictions apply.

applying a simple but statistically sound rule to the distribution of point-to-centroid distances. This yields an algorithm with automatic outlier detection and avoidance, named k-means-sharp (k-means#).

Some background on the classical k-means is provided in Section II. In Section III, we review existing approaches to dealing with outliers in k-means clustering, before discussing our modified algorithm in Section IV. Section V evaluates performance differences between the classical algorithm and ours, before the paper is concluded in Section VI.

II. K-MEANS CLUSTERING

K-means partitions the data space into Voronoi cells. Its goal is to form disjoint groups of n data points ($\mathbf{x}_1, \mathbf{x}_2, \ldots, \mathbf{x}_n$) into $k \leq n$ sets $\{S_1, S_2, \ldots, S_k\}$ that minimize the total mean within-cluster squared point-to-centroid distance or simply mean squared error. Thus, the optimization objective is to find:

$$\underset{S}{argmin} \frac{1}{n} \sum_{i=1}^{k} \sum_{x \in S_i} \|x - \mu_i\|^2 \quad (1)$$

where μ_i is the mean of points in S_i.

This, NP-hard problem, is widely handled using a heuristic (the classical k-means) which proceeds as follows:

1. Initialization: randomly select an initial set of cluster centroids.

2. Assignment: assign each point to the same cluster as the nearest centroids to that point. That is,

$$S_i^{(t)} = \{x: \|x - \mu_i^{(t)}\|^2 \leq \|x - \mu_j^{(t)}\|^2 \forall j, 1 \leq j \leq k\} \quad (2)$$

$S_i \cap S_j = \emptyset \; \forall \; i, j \leq k$. This implies that even if a point is equidistant to more than one centroid, it can only be assigned to one of them.

3. Centroid Update: Compute new centroids as the means of each cluster.

$$\mu_i^{(t+1)} = \frac{1}{|S_i^{(t)}|} \sum_{x_j \in S_i} x_j \quad (3)$$

4. Stop, if the means do not change i.e $\mu_i^{(t+1)} = \mu_i^{(t)} \forall i \leq k$ (which means the cluster assignments will not change on update) or if maximum iterations is reached. Otherwise, go to step 2.

As earlier mentioned, although the classical algorithm is popular and effective, the presence of outliers skews the partitioning produced.

III. RELATED WORK

Several techniques have been developed to deal with outliers in k-means. Some are based on seeding techniques which prevent outliers from being chosen as initial centroids, while others detect outliers so that they can be treated appropriately. One popular seeding technique is that of k-means++ [16]. A few years after k-means++ was published, Bahmani et.al. proposed a more scalable alternative named 'Scalable k-means++' [17]. Effective seeding produces robustness to initialization but does not remedy the false large values that outliers contribute to the objective function. This paper focuses on the outlier detection approach. Although traditional methods detect outliers in a module that is separate from the actual clustering process, our method performs clustering and outlier detection simultaneously.

The density-based approach of Breunig et.al. [18] is one popular outlier detection method. It computes a local outlier factor (LOF) for each point. The LOF of a point is based on the ratios of the local density of the area around that point and the local density of its neighbours. The LOF approach relies on a user-supplied minimum number of points, which is used to determine the neighbourhood size. Papadimitriou et.al. propose a similar technique called local correlation integral (LOCI) [19], to address the difficulty of choosing the minimum number of points in the LOF-based approach. It uses statistical values derived from the data itself. Murugavel and Punithavalli [20] observe that both the LOF and LOCI approaches scale poorly.

Another class of outlier detection techniques are based on k-nearest neighbor (kNN) graphs. Mutual kNN (MkNN) [21] uses a special case of the kNN graph. It defines an undirected proximity graph, which has a single edge between vertices. This approach helps distinguish inliers from outliers because inliers tend to be connected components in the graph. However, Hautamaki et.al. identified a problem with this approach [22]. They observe that it is ineffective in detecting an outlier that is close to an inlier. They then propose a preprocessing method which performs outlier detection using indegree number (ODIN) based on k-nearest neighbor graph. They define an outlier as a point that participates within T neighborhoods in the kNN graph, T being a control parameter. The graph is a directed proximity graph in which the points are vertices and the edges represent inter-point distances. In [23] another method is proposed, which sorts all points by their average kNN distances and defines a global threshold, such that points with large average distance are classified as outliers. The method assumes the number of outliers is known. This assumption is not often satisfied in practice and is especially unrealistic in the context of large datasets.

To get a better estimate of neighbourhood densities than classical nearest neighbor approaches, Jin et.al. propose the use of both of nearest neighbors and reverse nearest neighbours [15]. The reverse nearest neighbours of a point are points that have that particular point as their k nearest neighbours. A post-processing method has also been proposed, which refines k-means' initial result [24]. The method named outlier removal clustering (ORC) first gets initial centroids from running pure k-means until convergence. Then it iteratively removes points that are far from their centroids based on degree of 'outlyingness' computed for each point. A point has high outlyingness degree if this value exceeds a threshold T. Also worthy of mention, is the recent spectral clustering based outlier detection method of Wang et.al. [25].

Although these existing methods have been shown to be effective, they generally introduce significant departures from

Authorized licensed use limited to: INDIAN INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on October 29,2026 at 14:37:17 UTC from IEEE Xplore. Restrictions apply.

the standard k-means algorithm or increased computational cost which easily goes with poor scalability. Also, many methods rely on user-supplied parameters. In this paper, we seek to avoid these trade-offs. Our approach does not require construction of graphs and is easy to implement. Although our method is threshold-based like some of those mentioned, our threshold is a single global value that is computed online, without user intervention. The implication of a method like ours is that the classical k-means can be improved without incorporating pre-processing or post-processing, since outlier detection/avoidance is inherent to the algorithm, and is done simultaneously with the clustering process. Our algorithm, k-means#, preserves k-means simplicity and efficiency by maintaining the basic structure, altering only the centroid update step.

IV. MODIFIED ALGORITHM: K-MEANS SHARP (K-MEANS#)

K-means#, the modified algorithm proposed in this paper, is based on the rationale that the misleading effects of outliers in classical k-means happens at the centroids update stage. The new centroid of each cluster is computed as the cluster mean, a statistic that is sensitive to outliers. In this section, we explain how k-means# detects putative outliers in each iteration in order to avoid them when computing the means.

A. Outlier detection

K-means, being an instance of the Gaussian Mixture Model (GMM), assumes Gaussian data distribution [20][26]. It then follows that nearly all inliers (precisely 99.73%) will have point-to-centroid distances within 3 standard deviations (σ) from the population mean. This percentage is the area under the normal curve within $\pm 3\sigma$ from the mean. Therefore, we consider as outlier, any point at a distance outside this range from its cluster centroid.

It has been shown that $\sigma = 1.4826\ MAD_e$ is an unbiased estimate of the population standard deviation [27], where MAD_e = median $|e_i - \text{median}(e_i)|$.

Thus, the point-to-centroid distance threshold T which distinguishes inliers from outliers is given by:

$$T = 3\sigma$$
$$T = 3(1.4826\ MAD_e)$$
$$T = 4.4478 MAD_e \quad (4)$$

Note that T is a single global value that applies across all clusters.

B. New Centroid Update

Given a suitable value of T computed according to equation (4) in place of the standard update formula in (3), each centroid is updated as follows:

$$\mu_i^{(t+1)} = \frac{1}{|S_i^{(t)}| - |S_{io}^{(t)}|} \sum_{x_j \in S_i} x_j : x_j = \mathbf{0}\ if\ \|x - \mu_i\| > T \quad (5)$$

where $S_{io}^{(t)} \in S_i^{(t)}$ is the set of points x_j in $S_i^{(t)}$ that have point-to-centroid distance $\|x - \mu_i\| \geq T$.

The centroid computation equations (3) and (5) are similar. Both compute a new centroid as the mean of points in the same cluster. However, the difference is that in (5), each point that is not within distance T from its cluster centroid, is excluded from the computation.

C. Impact on cost function

The new cost function is a generalization, of which the standard k-means function is a specific case. Our algorithm excludes outliers from the partitioning, thus, they do not contribute to the mean squared error which k-means minimizes. So the classical k-means represents a special case in which all points are considered to be inliers.

The new cost function is given in equation (6) below, where n and n_0 are the data size and number of outliers respectively:

$$C = \frac{1}{n - n_0} \sum_i \rho\,(e_i^2) \quad (6)$$

$$\rho(\boldsymbol{e_i^2}) = \begin{cases} \|x - \mu_i\|^2 & \|x - \mu_i\| < \boldsymbol{T} \\ 0 & \|x - \mu_i\| \geq \boldsymbol{T} \end{cases} \quad (7)$$

D. Dynamic data and non-monotonically decreasing cost function

Our modification causes the inliers set (points treated as 'relevant data') to change in every iteration. Thus, the within-cluster error is no longer monotonically decreasing as it is in classical k-means, although preliminary experiments not reported in this paper, show that the within-cluster mean squared point-to-centroid distance signal eventually reaches stability. In our particular implementation, we take the 'best so far' as the final solution i.e. the clustering with the minimum error. For the datasets tested in Section V, 50 iterations proves to be sufficient.

E. The modified algorithm

The modified k-means algorithm (k-means#) is outlined as follows:

1. Select initial centroids as in k-means.
2. Cluster points around the centroids, as in k-means and compute corresponding cost C using equation (6).
3. Compute T using equation (4).
4. Update centroids using equation (5).
5. Stop, if no change in clustering or if maximum iterations is reached. Otherwise, go to step 2.
6. Return as final solution, the partition with minimum C.

V. EXPERIMENTS

The aim of the experiments reported in this section is to evaluate the impact of the proposed modification on the classical k-means. We compare mean square error (as a measure of clustering quality) of k-means and k-means# on 10 datasets. Furthermore, we evaluate k-means#'s performance in detecting outliers, based on accuracy, precision, recall and F1-score. We use 2 standard datasets (Iris and Ruspini) and 4 outlier-contaminated sets of each dataset created by adding

Authorized licensed use limited to: INDIAN INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on October 29,2026 at 14:37:17 UTC from IEEE Xplore. Restrictions apply.

gross errors to 5%, 10%, 15%, and 20%, of points in the datasets. This makes a total of 10 datasets: 2 free from outliers while 8 contain outliers.

A. Datasets

In each dataset, the true number of clusters (k), is known. The *Iris* dataset consists of 150 observations, 4 features and 3 classes ($k = 3$). The *Ruspini* dataset consists of 4 groups ($k = 4$) of 75 observations represented by 2 features. To construct the contaminated sets, the magnitudes of errors added are informed by the range of values in each dataset. For *Iris*, the errors added to each feature are integer values sampled from the set $\{r: 3 \leq r \leq 5\}$. For *Ruspini*, we sample from the set $\{r: 50 \leq r \leq 100\}$.

B. Results

Table 1 compares the average error over 100 runs of k-means# and k-means for each dataset. Each algorithm is allowed 50 iterations. In each of the 100 runs in the experiment, a single set of initial centroids is randomly selected to initialize both algorithms identically. Table 2 shows k-means#'s accuracy, precision, recall and F1-scores for the outlier-contaminated datasets. K-means is not included in Table 2 since it cannot detect outliers.

C. Discussion

The reported results show that, given identical initializations, k-means# yields better (i.e. lower) error than k-means for all datasets tested (Table 1, and Figures 1 and 2). This is due to k-means# ability to filter outliers and exempt them from formation of clusters. In contrast, k-means cannot distinguish outliers from inliers.

Table 2 summarizes evaluation of k-means#'s effectiveness for outlier detection. K-means is not included in the table because it has no outlier detection ability. The values in the table represent average performances on each dataset over 100 runs. The experiments were automated, making so many runs feasible. We note that there were rare cases among the 100 runs for some of the datasets in which k-means# returned zero true positives. Zero true positive implies undefined precision and F1 score. The problem could be caused by a bad set of randomly selected initial centroids, such as when all initial centroids are outliers. This drawback is inherited from k-means, and already mentioned in Section I. It can be addressed by incorporating special seeding methods into the algorithm, such as those discussed in Section III. Since such cases are rare in our experiments, we simply treat them as missing data and exclude them in the computation of the average values reported in Table 2. Excluding these rare cases poses no significant threat to statistical validity because reliable averages can be calculated from much less number of runs than 100.

Accuracy and precision have overall average of 90.4% and 86.8% respectively. Overall recall and F1-score are 52.4% and 61.1% respectively. The maximum values of these metrics achieved by the algorithm are 99.2%, 100%, 90% and 92.9%, respectively. High accuracy indicates a high percentage of data points are correctly classified with equal emphasis on inliers and outliers. High precision means a high percentage of points classified as outliers are indeed outliers. In other words, when the algorithm says a point is an outlier, it is often correct. On the other hand, recall measures the fraction of true outliers that the algorithm identifies as outliers. The F1-score combines both precision and recall.

TABLE I. COMPARING ERRORS OF K-MEANS AND K-MEANS#

Dataset (with outlier rate in %)	k-means	k-means#
Iris (0%)	0.91	0.54
Iris (5%)	2.60	1.01
Iris (10%)	4.78	1.78
Iris (15%)	3.42	2.36
Iris (20%)	3.51	2.73
Ruspini (0%)	593.79	438.67
Ruspini (5%)	949.64	606.53
Ruspini (10%)	977.48	884.44
Ruspini (15%)	1418.16	892.78
Ruspini (20%)	1244.43	1061.21

TABLE II. K-MEANS#'S OUTLIER DETECTION

Dataset (with outlier rate in %)	Accuracy (%)	Precision (%)	Recall (%)	F1-score (%)
Iris (5%)	99.2	97.3	90.0	92.9
Iris (10%)	97.9	98.8	80.1	89.1
Iris (15%)	93.0	100.0	45.1	63.4
Iris (20%)	82.7	99.9	13.5	24.2
Ruspini (5%)	89.3	48.9	70.8	53.2
Ruspini (10%)	86.3	55.0	57.6	53.4
Ruspini (15%)	90.4	94.3	38.7	72.1
Ruspini (20%)	84.7	99.8	23.3	40.6
Overall mean	**90.4**	**86.8**	**52.4**	**61.1**

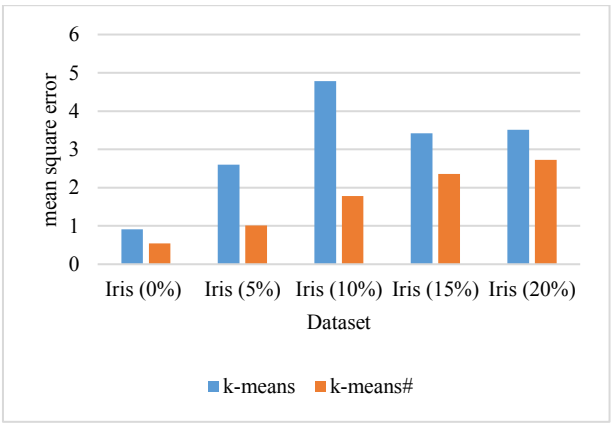


Fig. 1. Pure and contaminated Iris datasets: average error over 100 runs per dataset

Authorized licensed use limited to: INDIAN INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on October 29,2026 at 14:37:17 UTC from IEEE Xplore. Restrictions apply.

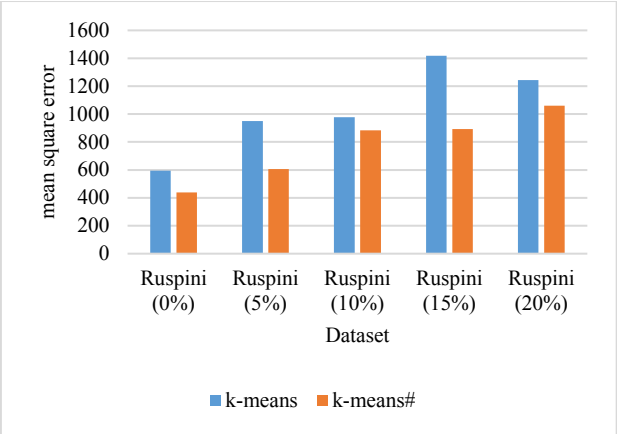


Fig. 2. Pure and contaminated Ruspini datasets: average error over 100 runs per dataset

The results summarize the performance improvements achieved due to our proposed modification. Unlike existing threshold-based approaches (see Section III), our threshold need not be supplied by the user. It is computed online using a statistically sound rule (See Section IV). It is expected that our choice of 3 standard deviations as the boundary for inlier point-to-centroid distance, plays a role in the performance of the algorithm. We expect trade-offs among the various performance metrics as this value is varied. In summary, in its ability to detect outliers, the k-means# implementation presented in this paper is highly accurate (90.4% average), highly precise (86.8% average) and moderately sensitive (52.4% average recall).

VI. Conclusion

This paper addressed k-means' sensitivity to outliers while maintaining the algorithm's basic structure, to preserve simplicity and efficiency. Specifically, we presented a new clustering algorithm named k-means# (pronounced k-means sharp), which modifies only the centroid update step in the original algorithm. In each iteration, putative outliers are detected automatically and excluded from the computation of new centroids. The new cost function is a generalization of the standard k-means', for which the latter represents a special case in which all points are considered inliers. The new algorithm preserves the ease of implementation for which the classical k-means is known.

We evaluated the impact of this simple modification on the original algorithm. In all cases tested, given identical initializations, k-means# yields better error than k-means. Furthermore, on the overall, its outlier detection mechanism exhibits high (90.4% average) accuracy, high (86.8% average) precision, with moderate recall and F1 score (52.4% and 61.1% averages), in contrast to classical k-means is unable to detect outliers. The algorithm achieves maximum values of 99.2%, 100%, 90% and 92.9%, respectively, of the aforementioned metrics.

A worthwhile direction for future work will be to investigate the sensitivity of the algorithm (in terms of each performance metric discussed) to the choice of the number of standard deviations for determining the inlier-outlier distinction threshold. Furthermore, the use of cluster-specific thresholds in contrast to a single global threshold, may improve performance, and is therefore worth exploring, although this will introduce some increase in computational cost. Lastly, since k-means# preserves the original k-means structure, it can easily benefit from many existing k-means improvements designed to address other drawbacks. Incorporation of some of these should yield increased robustness.

References

[1] G. Gan and M. Ng, "k-means clustering with outlier removal," *Pattern Recognition Letters*, 2017.

[2] X. Wu, V. Kumar, J. Quinlan, J. Ghosh, and Q. Yang, "Top 10 algorithms in data mining," *Knowledge and information systems*, 2008.

[3] S. Khanmohammadi and N. Adibeig, "An improved overlapping k-means clustering method for medical applications," *Expert Systems with Applications*, 2017.

[4] A. Jain, "Data clustering: 50 years beyond K-means," *Pattern recognition letters*, 2010.

[5] G. Gan, M. Kwok, and -Po Ng, "k -means clustering with outlier removal," *Pattern Recognition Letters*, vol. 90, pp. 8–14, 2017.

[6] J. Pena, J. Lozano, and P. Larranaga, "An empirical comparison of four initialization methods for the k-means algorithm," *Pattern recognition letters*, 1999.

[7] M. Steinbach and G. Karypis, "A comparison of document clustering techniques," *KDD workshop on text*, 2000.

[8] R. Kuo, L. Ho, and C. Hu, "Integration of self-organizing feature map and K-means algorithm for market segmentation," *Computers & Operations Research*, 2002.

[9] A. Chitade and S. Katiyar, "Colour based image segmentation using k-means clustering," *International Journal of Engineering Science and Technology* 2, no. 10 (2010): 5319-5325, 2010.

[10] I.T Afolabi, A.A Ezenwoke and C.K. Ayo, "Competitive analysis of social media data in the banking industry." *International Journal of Internet Marketing and Advertising* 11, no. 3: 183-201, 2017.

[11] A. Coates and A. Ng, "Learning feature representations with k-means," *Neural networks: Tricks of the trade*, 2012.

[12] A. Abbasi and M. Younis, "A survey on clustering algorithms for wireless sensor networks," *Computer communications*, 2007.

[13] P. Arora, D. Virmani, H. Jindal, and M. Sharma, "Sorted K-Means Towards the Enhancement of K-Means to Form Stable Clusters," *Proceedings of International Conference on Communication Networks,* Springer, Singapore, pp. 479–486, 2017.

[14] S.-H. Jung, K.-J. Kim, E.-C. Lim, and C.-B. Sim, "A Novel on Automatic K Value for Efficiency Improvement of K-means Clustering," *Proceeding of Internatioal Conference on Multimedia and Ubiquitous Engineering,* Springer, Singapore, pp. 181–186, 2017.

[15] Jin, Wen, Anthony KH Tung, Jiawei Han, and Wei Wang. "Ranking Outliers Using Symmetric Neighborhood Relationship." In *PAKDD*, vol. 6, pp. 577-593. 2006.

[16] Arthur, David, and Sergei Vassilvitskii. "k-means++: The advantages of careful seeding." In *Proceedings of the eighteenth annual ACM-SIAM Symposium on Discrete Algorithms*, pp. 1027-1035. Society for Industrial and Applied Mathematics, 2007.

[17] B. Bahmani, B. Moseley, A. Vattani, and R. Kumar, "Scalable k-means++," *Proceedings of the VLDB Endowment* 5, no. 7 (2012): 622-633., 2012.

[18] M. Breunig, H. Kriegel, R. Ng, and J. Sander, "LOF: identifying density-based local outliers," In *ACM sigmod record*, vol. 29, no. 2, pp. 93-104. ACM, 2000.

[19] S. Papadimitriou and H. Kitagawa, "Loci: Fast outlier detection using the local correlation integral," In P*roceedings of 19th International Conference on Data Engineering*, pp. 315-326. IEEE, 2003.

[20] P. Murugavel and M. Punithavalli, "Performance Evaluation of Density-

Authorized licensed use limited to: INDIAN INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on October 29,2026 at 14:37:17 UTC from IEEE Xplore. Restrictions apply.

Based Outlier Detection on High Dimensional Data." *International Journal on Computer Science and Engineering* 5, no. 2 (2013): 62, 2013.

[21] M. Brito, E. Chavez, A. Quiroz, and J. Yukich, "Connectivity of the mutual k-nearest-neighbor graph in clustering and outlier detection," *Statistics & Probability Letters*, 1997.

[22] V. Hautamaki and I. Karkkainen, "Outlier detection using k-nearest neighbour graph," *Pattern Recognition*, 2004.

[23] S. Ramaswamy, R. Rastogi, and K. Shim, "Efficient algorithms for mining outliers from large data sets," *ACM Sigmod Record*, 2000.

[24] V. Hautamäki, S. Cherednichenko, I. Kärkkäinen, T. Kinnunen, and P. Fränti, "Improving K-Means by Outlier Removal," In *Scandinavian Conference on Image Analysis*, pp. 978-987. Springer, Berlin, Heidelberg, 2005.

[25] Y. Wang, X. Wang, and X. Wang, "A Spectral Clustering Based Outlier Detection Technique." In *Machine Learning and Data Mining in Pattern Recognition*, pp. 15-27. Springer International Publishing, 2016.

[26] G. Hamerly and C. Elkan, "Learning the k in k-means," In *Advances in neural information processing systems*, pp. 281-288. , 2004.

[27] P. Rousseeuw and C. Croux, "Alternatives to the median absolute deviation." *Journal of the American Statistical association* 88, no. 424 (1993): 1273-1283. 1993.

Authorized licensed use limited to: INDIAN INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on October 29,2026 at 14:37:17 UTC from IEEE Xplore. Restrictions apply.